Relatório - Aplicação do Teorema Mestre

Máximo em matriz, soma de vetor com cópias e distâncias quadráticas entre pares

Disciplina: Análise de Algoritmos - UCB

Professor: Prof. Zoé Magalhães

Grupo: 1

Data: 29 de outubro de 2026

Autores: Victor Kauê e Gabriel Cardoso

Código-fonte e dados: https://github.com/VictorKaue/Teorema-Mestre

1. Introdução

A análise de algoritmos recursivos relaciona o custo total ao número de subproblemas, ao tamanho de cada subproblema e ao trabalho realizado fora das chamadas recursivas. O Teorema Mestre permite obter a ordem assintótica de diversas recorrências desse tipo, mas a análise deve corresponder às operações realmente executadas pelo código. Este trabalho aplica o teorema a três algoritmos recursivos, demonstra os resultados por indução e os compara com tempos medidos.

1.1 Objetivos

O objetivo geral é aplicar o Teorema Mestre e verificar a compatibilidade entre suas previsões e o crescimento dos tempos observados. Os objetivos específicos, detalhados para cada problema na seção 3.1, são: implementar as soluções recursivas em C; justificar o enquadramento de cada recorrência; demonstrar os resultados por indução; validar a correção por referências independentes; e registrar todas as amostras em um único arquivo CSV.

A tabela antecipa os resultados demonstrados nas seções 4.1 a 4.3.

Problema	Recorrência	Caso e complexidade
1. Máximo em matriz quadrada	$4T(n/2) + \Theta(1)$	Caso 1: $\Theta(n^2)$
2. Soma de vetor com cópias	$3T(n/3) + \Theta(n)$	Caso 2: $\Theta(n \log n)$
3. Distâncias quadráticas entre pares	$2T(n/2) + \Theta(n^2)$	Caso 3: $\Theta(n^2)$

Restrições do enunciado. Os Casos 1 e 2 exigem $a \neq 2$, e pelo menos um deles deve ter $a \neq b$. No Caso 1, $a = 4 \neq 2$ e $a \neq b$ ($4 \neq 2$); no Caso 2, $a = 3 \neq 2$ (aqui $a = b = 3$). As duas restrições são atendidas, e os três problemas são distintos entre si. Para manter divisões exatas, usam-se potências de 2 nos Casos 1 e 3 e potências de 3 no Caso 2.

2. Revisão teórica

O Teorema Mestre [1] resolve recorrências de divisão e conquista. Em uma árvore de recursão, o nível i tem a^i chamadas de tamanho n/b^i, e há $\log_b n$ níveis; as folhas somam $n^{\log_b a}$ chamadas. O teorema indica qual parte da árvore domina o custo: as folhas (Caso 1), todos os níveis por igual (Caso 2) ou a raiz (Caso 3).

Hipóteses. $a \geq 1$ e $b > 1$ são constantes; $f(n) \geq 0$; n/b é entendido como o piso ou o teto de n/b; $T(n) = \Theta(1)$ para n pequeno. Nas demonstrações da seção 4, n é potência de b, de modo que as divisões são exatas. Notação: $f(n) = O(g(n))$ significa $f(n) \leq c \cdot g(n)$ para alguma constante $c > 0$ e todo n suficientemente grande; Ω inverte a desigualdade; Θ vale quando O e Ω valem juntos.

Forma geral e função de referência

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$p = \log_b(a) \quad \Longrightarrow \quad g(n) = n^p$$

O logaritmo responde a que expoente devemos elevar b para obter a (por exemplo, $\log_2 4 = 2$, pois $2^2 = 4$). Comparamos o crescimento de $f(n)$ com o de $g(n) = n^p$.

Caso 1 - Recursão dominante

$$f(n) = O(n^{p-\varepsilon})$$

Hipótese: existe uma constante $\varepsilon > 0$ com esta propriedade.

$$T(n) = \Theta(n^p)$$

O trabalho local cresce polinomialmente menos que a referência. A maior contribuição vem das muitas chamadas próximas às folhas.

Caso 2 - Equilíbrio

$$f(n) = \Theta(n^p)$$

$$T(n) = \Theta(n^p \log n)$$

O trabalho total por nível tem a mesma ordem. Somar os $\log_b n$ níveis acrescenta o fator logarítmico.

Caso 3 - Trabalho não recursivo dominante

$$f(n) = \Omega(n^{p+\varepsilon})$$

Hipótese 1: existe uma constante $\varepsilon > 0$ com esta propriedade.

$$af\left(\frac{n}{b}\right) \leq qf(n)$$

Hipótese 2 (regularidade): existem $0 < q < 1$ e n_0 tais que vale para todo $n \geq n_0$.

$$T(n) = \Theta(f(n))$$

O trabalho local cresce polinomialmente mais que a referência, e a regularidade garante que o trabalho decresce a cada nível, de modo que a raiz domina. Para $f(n) = c \cdot n^k$ com $k > p$, a regularidade vale com $q = a/b^k < 1$; se f tem também termos de ordem menor, eles só afetam n pequeno, e a desigualdade vale para n grande. Basta calcular $a \cdot f(n/b)$ e compará-lo a $f(n)$.

Identidade das folhas. Para a > 0 e b > 1, a elevado a $\log_b n$ é igual a n elevado a $\log_b a$, porque o logaritmo na base b dos dois lados vale o produto $\log_b a \cdot \log_b n$. Por isso o número de folhas da árvore de recursão, a elevado a $\log_b n$, é n elevado a p, com $p = \log_b a$; é essa a razão de comparar f(n) com n^p.

Como aplicar. (i) Ler a, b e f(n) no algoritmo: quantas chamadas, que fração do tamanho cada uma recebe e qual o trabalho fora delas. (ii) Calcular $p = \log_b a$ e a referência n^p. (iii) Comparar f(n) com n^p: menor por um fator polinomial (Caso 1), da mesma ordem (Caso 2) ou maior por um fator polinomial com regularidade (Caso 3). Os três casos não cobrem todas as funções f; quando nenhum se aplica, o teorema não conclui.

3. Metodologia

3.1 Objetivo

O objetivo geral é aplicar o Teorema Mestre a três recorrências, demonstrar os resultados por indução e compará-los com tempos medidos de implementações em C. Os objetivos específicos de cada problema são:

Problema	Objetivo específico
1. Máximo em matriz (Caso 1)	Enquadrar $4T(n/2) + \Theta(1)$ no Caso 1 ($a = 4 \neq 2$ e $a \neq b$), demonstrar $\Theta(n^2)$ em função do lado e verificar se a medição acompanha o crescimento quadrático.
2. Soma com cópias (Caso 2)	Enquadrar $3T(n/3) + \Theta(n)$ no Caso 2 ($a = 3 \neq 2$), demonstrar $\Theta(n \log n)$ e verificar se o fator logarítmico é visível na faixa $9 \leq n \leq 729$.
3. Distâncias entre pares (Caso 3)	Enquadrar $2T(n/2) + \Theta(n^2)$ no Caso 3, verificar a regularidade, demonstrar $\Theta(n^2)$ e comparar com a medição.

3.2 Ferramentas utilizadas

Código em C11, compilado com Apple clang 21.0.0 em macOS 27.0.1, opções -O2 -Wall -Wextra -pedantic. Desempenho: clock_gettime(CLOCK_MONOTONIC), relógio monotônico de tempo decorrido (no Windows, o mesmo código usa QueryPerformanceCounter). Máquina: MacBook com chip Apple M4 (modelo Mac16,1), com 10 núcleos (4 de desempenho e 6 de eficiência) e 24 GiB de memória; segundo o sysctl, L1 de dados de 64 KiB, L1 de instruções de 128 KiB e L2 de 16 MiB no cluster de desempenho e de 4 MiB no de eficiência. Tabelas, gráficos e relatório: Python, com ReportLab (tabelas e PDF) e Matplotlib (gráficos e fórmulas). Os tempos caracterizam apenas esta máquina e não permitem comparações com outras.

3.3 Métodos

Coleta dos dados

O programa gera inteiros entre -10 e 10 com um gerador congruencial linear determinístico de semente 20261028; a semente reproduz os dados quando os mesmos parâmetros são usados. O máximo recebe uma matriz quadrada armazenada em um vetor; a soma recebe um vetor de inteiros; a soma de distâncias recebe um vetor de coordenadas inteiras (os algoritmos são descritos na seção 4).

Foram usados n = 16, 32, 64, 128, 256, 512 e 1024 nos Casos 1 e 3 (no primeiro, n é o lado da matriz, contendo n^2 elementos); e n = 9, 27, 81, 243 e 729 no Caso 2. Cada combinação foi medida em cinco amostras. Em cada amostra, o número de repetições dobra até o lote durar pelo menos 25 ms, limitado a 1.048.576 repetições; as tentativas de calibração anteriores ao lote final não entram no CSV. O tempo médio por execução é:

Cálculo

$$\bar{t}_{\text{ms}} = \frac{1000\,(t_{\text{fim}} - t_{\text{inicio}})}{r}$$

Na fórmula, t_{fim} e $t_{início}$ são os instantes de fim e de início do lote, medidos em segundos; r é o número de repetições do lote; e o fator 1000 converte segundos em milissegundos. Nas tabelas, ns/g(n) é o tempo mediano em nanossegundos (o tempo em ms multiplicado por 10^6) dividido por g(n). O CSV registra caso, tamanho, amostra, repetições, tempo médio, métrica analítica, referência assintótica e validação: 95 amostras para 19 combinações de problema e tamanho (Apêndice A). As tabelas usam a mediana das cinco médias de lote, além do mínimo e do máximo.

Validação da correção e das contagens

Antes dos lotes, o máximo é comparado com uma varredura da matriz, a soma com um laço simples e as distâncias com dois laços diretos. Um programa adicional aprovou 102 verificações com casos base, zeros, valores negativos, repetições, vetores crescentes e máximos em diferentes posições. As contagens analíticas do CSV ($n^2 - 1$ comparações, $n \cdot \log_3 n$ cópias e $n(n - 1)/2$ pares) foram conferidas por uma cópia instrumentada dos algoritmos, que também confirmou o número de chamadas, $(4n^2 - 1)/3$ no Caso 1 e $(3n - 1)/2$ no Caso 2. A geração de dados, as referências e a escrita do CSV ficam fora da região cronometrada.

Análise e comparação com as previsões teóricas

Para cada problema, o tempo mediano é normalizado por g(n) (n^2, $n \cdot \log_3 n$ e n^2, respectivamente). Se a previsão vale, a razão ns/g(n) tende a uma constante. Também se comparam as razões de crescimento entre tamanhos consecutivos com as previstas pela teoria e se estima o expoente empírico, isto é, a inclinação da regressão linear de log t sobre log n. Os gráficos mostram o tempo medido, a curva $C \cdot g(n)$, com C igual à média geométrica de t/g(n) (nenhum ponto é privilegiado), e, nos Casos 2 e 3, o modelo de custo S(n), definido no início da seção 4, ajustado por mínimos quadrados. Convergência significa concordância dentro de cada regime; divergência, um desvio sistemático, para o qual se discutem causas possíveis. A comparação é uma evidência da faixa medida, e a prova é a da seção 4.

3.4 Justificativa das escolhas

As potências de 2 e 3 evitam arredondamentos nas recorrências. O relógio de alta resolução e os lotes reduzem o efeito de execuções curtas; a mediana reduz a influência de extremos entre cinco amostras. C permite observar chamadas e cópias reais; Python facilita resumir o CSV e gerar gráficos reproduzíveis. As contagens instrumentadas separam a verificação da recorrência (quantas operações) da medição de tempo (quanto custam na máquina).

3.5 Limitações e fraquezas da metodologia

A soma inclui suas alocações, cópias e liberações internas, e o custo de malloc e free não é controlado. As entradas são geradas e alocadas antes da medição e não são alteradas pelos algoritmos; as repetições reutilizam os mesmos dados e favorecem cache e predição de desvios. O relógio mede tempo decorrido, sujeito a escalonamento e carga: no macOS não foi possível fixar o núcleo (de desempenho ou de eficiência) nem a frequência, e a coleta foi feita com o notebook na bateria. Cinco amostras não caracterizam toda a variabilidade do ambiente; um lote isolado (Caso 1, n = 128, amostra 4) ficou 45% acima da mediana do seu grupo, que a mediana absorve. Cada tamanho usa uma única entrada pseudoaleatória. A faixa do Caso 2 ($\log_3 n$ de 2 a 6) é estreita para separar n de n log n. A entrada do Caso 1 chega a 8 MiB, enquanto as dos Casos 2 e 3 cabem em poucos KiB; como não se sabe em qual núcleo cada lote rodou, efeitos de cache só podem ser levantados como hipótese. Não se mediram bytes de pilha ou heap.

4. Análise teórica

Modelo de custo S. O Teorema Mestre trabalha com Θ, que esconde constantes. Para demonstrar cada resultado por indução, usamos um modelo de custo S(n), obtido da recorrência de T(n) ao trocar cada Θ por uma constante: **c** representa o custo do trabalho local e **d** o custo do caso base. S(n) não é o tempo exato de CPU: representa o crescimento da implementação, supondo aritmética de inteiros de tamanho fixo e custo constante por operação elementar de memória. Em cada problema, primeiro aplicamos o teorema, depois resolvemos S(n) exatamente por indução e, por fim, mostramos que T(n) fica entre dois modelos S com constantes diferentes (trecho "Do modelo S para T").

4.1 Problema 1 - Maior valor de uma matriz

O problema pede o maior valor de uma matriz quadrada de lado n, ou seja, com N = n² elementos. A estratégia é dividir e conquistar: a matriz é partida em quatro quadrantes de lado n/2, cada quadrante é resolvido por uma chamada recursiva, e o maior dos quatro resultados é o máximo da matriz inteira. A função completa é mostrada abaixo; as quatro chamadas recursivas, uma por quadrante, usam a variável metade, que vale n/2:

```
int64_t maximo_matriz(const int64_t *matriz, int largura, int linha, int coluna, int n) {
    if (n == 1) return matriz[(size_t)linha * largura + coluna];
    int metade = n / 2;
    int64_t maior = maximo_matriz(matriz, largura, linha, coluna, metade);
    int64_t valor = maximo_matriz(matriz, largura, linha, coluna + metade, metade);
    if (valor > maior) maior = valor;
    valor = maximo_matriz(matriz, largura, linha + metade, coluna, metade);
    if (valor > maior) maior = valor;
    valor = maximo_matriz(matriz, largura, linha + metade, coluna + metade, metade);
    if (valor > maior) maior = valor;
    return maior;
}
```

Como ler o código. Para usar o Teorema Mestre, precisamos extrair da implementação três informações. (1) **a**, o número de chamadas recursivas: são quatro, uma para cada quadrante, logo a = 4. (2) **b**, a razão pela qual o tamanho diminui: cada chamada recebe um quadrante de lado n/2, logo b = 2 (aqui n é o lado da matriz, e não a quantidade de elementos). (3) **f(n)**, o trabalho feito fora das chamadas: depois de receber os quatro resultados, a função só faz três comparações (valor > maior) e alguns cálculos de índice, que não dependem de n; logo f(n) = Θ(1). Quando n = 1 não há recursão e a função devolve o único elemento, o que custa T(1) = Θ(1).

Correção. No caso base existe apenas um elemento, que já é o máximo. Supondo o algoritmo correto para o lado n/2, como os quatro quadrantes cobrem toda a matriz sem sobreposição, o maior dos quatro máximos é o máximo da matriz completa.

Resumo: do código à recorrência

Observação no código	Informação para o teorema
Quatro chamadas de maximo_matriz	a = 4
Cada chamada recebe metade do lado	b = 2
Três comparações e operações fixas por chamada	f(n) = Θ(1)

Com a = 4, b = 2 e f(n) = Θ(1), aplicamos o teorema em três passos: escrever a recorrência, calcular a função de referência n^p e comparar o trabalho local f(n) com ela.

1. Escrever a recorrência

$$T(n) = 4T\left(\frac{n}{2}\right) + \Theta(1)$$

$$T(1) = \Theta(1)$$

2. Calcular a função de referência

$$p = \log_2(4) = 2 \qquad 2^2 = 4$$

O logaritmo indica o expoente que transforma 2 em 4.

$$g(n) = n^p = n^2$$

Por que n^p é a referência: a árvore tem $\log_2 n$ níveis e 4 chamadas por nível, logo o número de folhas é 4 elevado a $\log_2 n$. Pela identidade da seção 2, isso é n elevado a $\log_2 4$, isto é, n² folhas, e cada folha custa Θ(1).

3. Comparar o trabalho local com a referência

$$f(n) = \Theta(1) \qquad g(n) = n^2$$

$$1 < n^2 \qquad (n > 1)$$

A constante cresce menos que a função quadrática.

$$\varepsilon = 2 \implies n^{p-\varepsilon} = n^{2-2} = 1$$

$$f(n) = O(n^{p-\varepsilon})$$

A comparação confirma o Caso 1: o trabalho local é polinomialmente menor. Aqui n é o lado; a matriz tem N = n² elementos. Logo, o resultado também pode ser escrito como Θ(N) em função da quantidade total de elementos; como todo algoritmo precisa ler os N elementos, esta solução é assintoticamente ótima em N.

Concluímos aplicando o Caso 1, cujo resultado é $T(n) = \Theta(n^p)$ com p igual ao logaritmo de a na base b. Substituindo a = 4 e b = 2:

Resultado - Caso 1: recursão dominante

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

Com a = 4 e b = 2: log□4 = 2, pois 2² = 4. O expoente de n é, portanto, 2.

4.1.1 Demonstração

O Teorema Mestre usa Θ, que esconde constantes. Para provar o resultado por indução, trocamos o Θ(1) por constantes concretas: **c** é o custo do trabalho local de uma chamada (as três comparações e os índices) e **d** é o custo do caso base (n = 1). Assim, o modelo S(n) = 4S(n/2) + c, com S(1) = d, tem solução exata, que obtemos por expansão e confirmamos por indução; depois mostramos que T(n) fica entre dois modelos desse tipo.

Derivação. Para obter a fórmula, expande-se a recorrência do modelo de custo S(n) = 4S(n/2) + c, com S(1) = d: após k = $\log_2 n$ níveis restam 4^k = n² folhas, e cada nível i contribui $4^i \cdot c$.

Expansão (k = log□n, $4^k = n^2$)

$$S(n) = 4^k d + c(1 + 4 + \cdots + 4^{k-1})$$
$$= n^2 d + c\frac{n^2 - 1}{3}$$
$$= \left(d + \frac{c}{3}\right)n^2 - \frac{c}{3}$$

Indução. Verificamos que a fórmula obtida vale para todas as potências de 2, com c e d constantes positivas.

Modelo de custo e fórmula que queremos provar

$$S(1) = d \qquad S(n) = 4S\left(\frac{n}{2}\right) + c$$
$$S(n) = \left(d + \frac{c}{3}\right)n^2 - \frac{c}{3}$$

Base - substituir n por 1

$$\left(d + \frac{c}{3}\right) \cdot 1^2 - \frac{c}{3} = d$$

Hipótese - admitir a fórmula para o tamanho menor

$$S\left(\frac{n}{2}\right) = \left(d + \frac{c}{3}\right)\left(\frac{n}{2}\right)^2 - \frac{c}{3}$$

Passo - substituir a hipótese e simplificar

$$S(n) = 4S\left(\frac{n}{2}\right) + c$$
$$= 4\left[\left(d + \frac{c}{3}\right)\left(\frac{n}{2}\right)^2 - \frac{c}{3}\right] + c$$
$$= \left(d + \frac{c}{3}\right)n^2 - \frac{4c}{3} + c$$
$$= \left(d + \frac{c}{3}\right)n^2 - \frac{c}{3}$$

Obtivemos a mesma fórmula proposta. Como ela vale para n = 1 e passa do tamanho menor ao maior, vale para todas as potências de 2. O coeficiente de n^2 é positivo e o termo restante é constante; portanto, $S(n) = \Theta(n^2)$.

Do modelo S para T. O Θ(1) da recorrência representa uma função f(n) desconhecida, com $c_1 \le f(n) \le c_2$, e T(1) fica entre d_1 e d_2. Como a recorrência é monotônica em f e em T(1), a indução em n dá $S_{c1,d1}(n) \le T(n) \le S_{c2,d2}(n)$. Pela fórmula acima, os dois limites são $\Theta(n^2)$; logo, $T(n) = \Theta(n^2)$, o mesmo resultado do Teorema Mestre. A demonstração vale para n potência de 2, que são os tamanhos que o código aceita.

Contagem de comparações e exemplo

A métrica do CSV conta somente comparações entre valores. Seu caso base é zero e cada chamada não trivial acrescenta três comparações:

Contagem exata, calculada pela fórmula

$$C(1) = 0 \qquad C(n) = 4C\left(\frac{n}{2}\right) + 3$$

$$4\left[\left(\frac{n}{2}\right)^2 - 1\right] + 3 = n^2 - 1$$

$$C(n) = n^2 - 1$$

Exemplo 4 × 4, com linhas [3, -1, 5, 0], [7, 2, -4, 1], [-2, 6, 8, -5] e [0, 4, -3, 9]. Os quatro quadrantes 2 × 2 devolvem 7, 5, 6 e 9; cada um usa três comparações, e três comparações finais selecionam 9: no total, 4·3 + 3 = 15 = 4² - 1. O espaço auxiliar é O(log n), devido à pilha; a matriz de entrada ocupa n² posições.

4.2 Problema 2 - Soma de vetor com cópias

O problema pede a soma dos n valores de um vetor. Esta versão, didática, divide o vetor em três terços de tamanho n/3, copia cada terço para um vetor próprio, soma os resultados das três chamadas recursivas e libera os vetores. A função completa é mostrada abaixo; o laço de cópia define f(n) e as três chamadas recursivas usam tamanho = n/3:

```
int64_t somar_vetor(const int64_t *vetor, int n) {
    if (n == 1) return vetor[0];
    int tamanho = n / 3;
    int64_t *parte1 = malloc((size_t)tamanho * sizeof(*parte1));
    int64_t *parte2 = malloc((size_t)tamanho * sizeof(*parte2));
    int64_t *parte3 = malloc((size_t)tamanho * sizeof(*parte3));
    if (!parte1 || !parte2 || !parte3) {
        fprintf(stderr, "Memoria insuficiente.\n");
        free(parte1); free(parte2); free(parte3);
        exit(EXIT_FAILURE);
    }
    for (int i = 0; i < tamanho; i++) {
        parte1[i] = vetor[i];
        parte2[i] = vetor[i + tamanho];
        parte3[i] = vetor[i + 2 * tamanho];
    }
    int64_t soma = somar_vetor(parte1, tamanho);
    soma += somar_vetor(parte2, tamanho);
    soma += somar_vetor(parte3, tamanho);
    free(parte1); free(parte2); free(parte3);
    return soma;
}
```

Como ler o código. (1) **a**: são três chamadas, uma para cada terço, logo a = 3. (2) **b**: cada chamada recebe um terço do vetor, logo b = 3. (3) **f(n)**: antes das chamadas, o laço mostrado acima percorre os n/3 índices de um terço e copia três valores por iteração; no total, n valores são copiados em cada chamada de tamanho n, além de três alocações e liberações. Esse trabalho é proporcional a n, logo f(n) = Θ(n). Para n = 1 a função devolve o único elemento: T(1) = Θ(1).

Correção. As três partes preservam todos os valores do vetor (n potência de 3), sem repetição nem perda; somar seus resultados produz a soma do vetor original. A escolha é didática: uma soma comum pode ser linear, enquanto as cópias reais desta versão geram trabalho em todos os níveis.

Resumo: do código à recorrência

Observação no código	Informação para o teorema
Três chamadas de somar_vetor	a = 3
Cada chamada recebe um terço do vetor	b = 3
O laço copia três valores por iteração, n no total	f(n) = Θ(n)

Com a = 3, b = 3 e f(n) = Θ(n), seguimos os mesmos três passos: recorrência, referência n^p e comparação.

> **1. Escrever a recorrência**
>
> $$T(n) = 3T\left(\frac{n}{3}\right) + \Theta(n)$$
>
> $$T(1) = \Theta(1)$$

2. Calcular a função de referência

$$p = \log_3(3) = 1 \qquad 3^1 = 3$$

O logaritmo indica o expoente que transforma 3 em 3.

$$g(n) = n^p = n$$

Por que n^p é a referência: a árvore tem $\log_3 n$ níveis e 3 chamadas por nível, logo o número de folhas é 3 elevado a $\log_3 n$. Pela identidade da seção 2, isso é n elevado a $\log_3 3$, isto é, n folhas.

3. Comparar o trabalho local com a referência

$$f(n) = \Theta(n) \qquad g(n) = n$$

$$n = n$$

As duas funções têm a mesma ordem de crescimento.

$$f(n) = \Theta(g(n))$$

As duas funções têm a mesma ordem de crescimento: é o Caso 2. O total copiado por nível é n e existem $\log_3(n)$ níveis de cópia. **Contraste:** sem as cópias (passando apenas índices), o trabalho local seria Θ(1) e a recorrência, 3T(n/3) + Θ(1), com p = 1 e f(n) = $O(n^{1-\varepsilon})$ para ε = 1: Caso 1, com T(n) = Θ(n). O mesmo esquema de chamadas muda de caso conforme o trabalho fora delas.

Concluímos aplicando o Caso 2, cujo resultado é T(n) = $\Theta(n^p \log n)$ com p igual ao logaritmo de a na base b. Substituindo a = 3 e b = 3:

Resultado - Caso 2: equilíbrio

$$T(n) = \Theta\left(n^{\log_b a} \log n\right) = \Theta\left(n^{\log_3 3} \log n\right) = \Theta\left(n^1 \log n\right) = \Theta(n \log n)$$

Com a = 3 e b = 3: log□3 = 1, pois $3^1 = 3$. O fator $n^1 = n$ multiplica o fator logarítmico do Caso 2.

4.2.1 Demonstração

Como no Problema 1, trocamos o Θ por constantes: **c** é o custo de copiar um elemento (de modo que o trabalho local é cn) e **d** é o custo do caso base. O modelo é S(n) = 3S(n/3) + cn, com S(1) = d. Custos que não dependem do tamanho (as três alocações e liberações de cada chamada) cabem em $c_1 n \le f(n) \le c_2 n$ para n ≥ 3; na comparação com os tempos medidos, eles são absorvidos pela constante efetiva d do ajuste.

Derivação. Para o modelo S(n) = 3S(n/3) + cn, com S(1) = d, há $k = \log_3 n$ níveis, $3^k = n$ folhas, e cada nível i soma 3^i chamadas de custo $c \cdot n/3^i$, isto é, cn por nível.

Expansão (k = log□n, $3^k = n$)

$$S(n) = 3^k d + \sum_{i=0}^{k-1} 3^i \cdot c \cdot \frac{n}{3^i}$$

$$= dn + c\,n\,k$$

$$= dn + cn\log_3(n)$$

Indução. Considere n potência de 3 e c, d positivos. A fórmula deve valer no caso base e continuar verdadeira quando combinamos os três subproblemas.

Modelo de custo e fórmula que queremos provar

$$S(1) = d \qquad S(n) = 3S\left(\frac{n}{3}\right) + cn$$

$$S(n) = dn + cn\log_3(n)$$

Base - substituir n por 1

$$d \cdot 1 + c \cdot 1 \cdot \log_3(1) = d$$

O logaritmo de 1 é zero; esse termo desaparece.

Hipótese - admitir a fórmula para o tamanho menor

$$S\left(\frac{n}{3}\right) = d\frac{n}{3} + c\frac{n}{3}\log_3\left(\frac{n}{3}\right)$$

Passo - substituir a hipótese e simplificar

$$\begin{aligned} S(n) &= 3S\left(\frac{n}{3}\right) + cn \\ &= 3\left[d\frac{n}{3} + c\frac{n}{3}\log_3\left(\frac{n}{3}\right)\right] + cn \\ &= dn + cn(\log_3(n) - 1) + cn \\ &= dn + cn\log_3(n) \end{aligned}$$

No terceiro passo, usamos $\log_3(n/3) = \log_3(n) - 1$. O resultado coincide com a fórmula proposta e confirma a indução. O termo $n \log_3(n)$ domina o termo linear; assim, $S(n) = \Theta(n \log n)$.

Do modelo S para T. Como no Problema 1, f(n) está entre $c_1 n$ e $c_2 n$ e T(1) entre d_1 e d_2. A monotonicidade da recorrência dá $S_{c1,d1}(n) \le T(n) \le S_{c2,d2}(n)$, e ambos os limites são $\Theta(n \log n)$; logo, $T(n) = \Theta(n \log n)$, como prevê o Teorema Mestre (n potência de 3).

Contagem de cópias e exemplo

Elementos copiados em todos os níveis

$$E(n) = n \cdot \log_3(n)$$

$$E(3) = 3 \cdot 1 = 3$$

$$E(9) = 9 \cdot 2 = 18$$

Para [2, -1, 4], o programa copia [2], [-1] e [4]. As chamadas-base retornam esses valores e a soma é 5. Para n = 9, são nove cópias na raiz e mais nove no nível seguinte. Os vetores ativos somam $n + n/3 + \ldots \le 3n/2$ posições: espaço auxiliar O(n), com pilha O(log n).

4.3 Problema 3 - Distâncias quadráticas entre pares

Dados n valores, o objetivo é somar o quadrado da diferença de cada par de posições distintas. O programa divide o vetor em duas metades e resolve cada uma por uma chamada recursiva, o que cobre os pares com os dois elementos na mesma metade. Os pares cruzados (um elemento em cada metade) não aparecem em nenhuma chamada e são enumerados por laços. Cada par é contado exatamente uma vez, no nó em que seus elementos ficam em metades diferentes. A função completa é mostrada abaixo; as duas chamadas recursivas cobrem as metades e os laços dos pares cruzados definem f(n) (com metade = n/2):

```
int64_t somar_distancias(const int64_t *vetor, int n) {
    if (n <= 1) return 0;
    int metade = n / 2;
    int64_t soma = somar_distancias(vetor, metade);
    soma += somar_distancias(vetor + metade, n - metade);
    for (int i = 0; i < metade; i++) {
        for (int j = metade; j < n; j++) {
            int64_t diferenca = vetor[i] - vetor[j];
            soma += diferenca * diferenca;
        }
    }
    return soma;
}
```

> **Quantidade que o algoritmo calcula**
>
> $$D = \sum_{i<j} (v_i - v_j)^2$$

Como ler o código. (1) **a**: são duas chamadas, uma para cada metade, logo a = 2. (2) **b**: cada chamada recebe metade do vetor, logo b = 2. (3) **f(n)**: depois das chamadas, os dois laços aninhados mostrados acima percorrem os índices i da primeira metade e j da segunda, avaliando (n/2)·(n/2) = n²/4 pares, cada um com custo constante (diferença, quadrado e soma); logo f(n) = Θ(n²). Para n ≤ 1 não há pares: T(1) = Θ(1).

Resumo: do código à recorrência

Observação no código	Informação para o teorema
Duas chamadas de somar_distancias	a = 2
Cada chamada recebe metade do vetor	b = 2
Os laços avaliam (n/2) · (n/2) pares cruzados	f(n) = Θ(n²)

Com a = 2, b = 2 e f(n) = Θ(n²), aplicamos o teorema: recorrência, referência n^p e comparação. No Caso 3 há um passo extra, a regularidade, que garante que o trabalho local diminui a cada nível.

> **1. Recorrência e função de referência**
>
> $$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n^2), \quad T(1) = \Theta(1)$$
>
> $$p = \log_2(2) = 1 \quad \Longrightarrow \quad g(n) = n$$

Por que n^p é a referência: a árvore tem $\log_2 n$ níveis e 2 chamadas por nível, logo o número de folhas é 2 elevado a $\log_2 n$. Pela identidade da seção 2, isso é n elevado a $\log_2 2$, isto é, n folhas (p = 1). Como f(n) = Θ(n²) cresce mais que n, o trabalho fora das chamadas supera o das folhas.

2. Comparar o crescimento

$$f(n) = \Theta(n^2) \qquad g(n) = n$$

$$n^2 > n \qquad (n > 1)$$

A função quadrática cresce mais que a linear.

$$\varepsilon = 1 \implies n^{p+\varepsilon} = n^2$$

$$f(n) = \Omega\left(n^{p+\varepsilon}\right)$$

3. Verificar a regularidade, usando f(n) = cn²

$$2f\left(\frac{n}{2}\right) = 2c\left(\frac{n}{2}\right)^2$$

$$= \frac{cn^2}{2} = \frac{f(n)}{2}$$

$$q = \frac{1}{2} < 1$$

O custo local real é cn² mais termos de ordem menor (a sobrecarga dos laços, proporcional a n). Esses termos alteram a desigualdade apenas para n pequeno: para n grande, 2f(n/2) = f(n)/2 + O(n) ≤ q·f(n) para qualquer q entre 1/2 e 1, por exemplo q = 3/4. A regularidade vale, portanto, para o f real, e não só para cn² exato.

O trabalho local cresce polinomialmente mais que a referência e satisfaz a regularidade. As duas exigências confirmam o Caso 3.

Concluímos aplicando o Caso 3: T(n) = Θ(f(n)). Comparando a referência calculada com f(n) e substituindo:

Resultado - Caso 3: trabalho não recursivo dominante

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n \; < \; n^2 = f(n)$$

Com a = 2 e b = 2: log□2 = 1, pois 2¹ = 2. A referência é n, e f(n) = n² é polinomialmente maior.

$$T(n) = \Theta\left(f(n)\right) = \Theta\left(n^2\right)$$

No Caso 3 (com a regularidade verificada, q < 1), o resultado é o próprio f(n).

4.3.1 Demonstração

Trocamos o Θ por constantes: **c** é o custo associado ao trabalho local, de modo que ele vale cn² (c absorve o fator 1/4 dos n²/4 pares e o custo de cada par) e **d** é o custo do caso base. O modelo é S(n) = 2S(n/2) + cn², com S(1) = d.

Derivação. Para o modelo S(n) = 2S(n/2) + cn², com S(1) = d, há k = $\log_2 n$ níveis e 2^k = n folhas; o nível i soma 2^i chamadas de custo $c(n/2^i)^2$, uma série geométrica de razão 1/2.

Expansão (k = log□n, $2^k = n$)

$$\begin{aligned} S(n) &= 2^k d + \sum_{i=0}^{k-1} 2^i \cdot c\left(\frac{n}{2^i}\right)^2 \\ &= dn + cn^2 \sum_{i=0}^{k-1} 2^{-i} \\ &= dn + cn^2\left(2 - \frac{2}{n}\right) \\ &= 2cn^2 + (d-2c)n \end{aligned}$$

Indução. Considere n potência de 2 e c, d positivos. Vamos provar a fórmula do modelo e, separadamente, contar os pares avaliados.

Modelo de custo e fórmula que queremos provar

$$S(1) = d \qquad S(n) = 2S\left(\frac{n}{2}\right) + cn^2$$

$$S(n) = 2cn^2 + (d-2c)n$$

Base - substituir n por 1

$$2c \cdot 1^2 + (d-2c) \cdot 1 = d$$

Hipótese - admitir a fórmula para o tamanho menor

$$S\left(\frac{n}{2}\right) = 2c\left(\frac{n}{2}\right)^2 + (d-2c)\frac{n}{2}$$

Passo - substituir a hipótese e simplificar

$$\begin{aligned} S(n) &= 2S\left(\frac{n}{2}\right) + cn^2 \\ &= 2\left[2c\left(\frac{n}{2}\right)^2 + (d-2c)\frac{n}{2}\right] + cn^2 \\ &= cn^2 + (d-2c)n + cn^2 \\ &= 2cn^2 + (d-2c)n \end{aligned}$$

A fórmula é preservada. O coeficiente de n² é positivo e o termo linear tem ordem inferior. Portanto, S(n) = Θ(n²). O espaço auxiliar é O(log n), correspondente à pilha de chamadas.

Do modelo S para T. Aqui f(n) está entre c_1n^2 e c_2n^2 e T(1) entre d_1 e d_2; a monotonicidade dá $S_{c1,d1}(n) \le T(n) \le S_{c2,d2}(n)$, ambos Θ(n²). Logo, T(n) = Θ(n²), coincidindo com o Caso 3 (n potência de 2).

Contagem exata de pares e exemplo

Seja P(n) o número de pares avaliados: P(1) = 0 e P(n) = 2P(n/2) + (n/2)², em que (n/2)² = n²/4 são os pares cruzados. Afirmamos que P(n) = n(n - 1)/2, e a mesma indução se aplica: P(1) = 0 confere com a fórmula; admitindo-a para n/2,

Hipótese - admitir a fórmula para o tamanho menor

$$P\left(\frac{n}{2}\right) = \frac{1}{2} \cdot \frac{n}{2}\left(\frac{n}{2} - 1\right)$$

Passo - substituir a hipótese na recorrência de P

$$\begin{aligned} P(n) &= 2P\left(\frac{n}{2}\right) + \frac{n^2}{4} \\ &= 2\left[\frac{1}{2}\left(\frac{n}{2}\right)\left(\frac{n}{2} - 1\right)\right] + \frac{n^2}{4} \\ &= \frac{n^2}{4} - \frac{n}{2} + \frac{n^2}{4} \\ &= \frac{n(n-1)}{2} \end{aligned}$$

Para [1, 3, 2, 4], as metades contribuem 4 + 4 e os pares cruzados contribuem 1 + 9 + 1 + 1. A soma é 20, com seis pares no total. Esta implementação enumera pares, mas o problema admite solução linear pela identidade abaixo (que se obtém desenvolvendo $(v_i - v_j)^2$ e somando sobre os pares), portanto $\Theta(n^2)$ não é um limite inferior do problema. Para o exemplo: $4 \cdot (1 + 9 + 4 + 16) - 10^2 = 120 - 100 = 20$.

Identidade algébrica

$$D = n\sum_i v_i^2 - \left(\sum_i v_i\right)^2$$

5. Resultados experimentais e comparação com a teoria

5.1 Problema 1 - Maior valor de uma matriz

A tabela traz a mediana das cinco médias de lote, o mínimo e o máximo, a contagem analítica de operações (conferida por instrumentação, seção 3.3) e a razão ns/g(n), com g(n) = n². A unidade ns/g(n) é apenas uma razão de escala, e não o custo exato de uma instrução. A coluna Entrada é o tamanho da matriz, n² posições de 8 bytes.

n (lado)	Mediana (ms)	Mín. a máx. (ms)	Comparações (fórmula)	ns / g(n)	Entrada (KiB)
16	0,000299	0,000294 a 0,000384	255	1,169	2
32	0,001212	0,001204 a 0,001221	1023	1,184	8
64	0,004934	0,004826 a 0,004981	4095	1,205	32
128	0,019894	0,019678 a 0,028859	16383	1,214	128
256	0,080809	0,080654 a 0,081951	65535	1,233	512
512	0,324203	0,322516 a 0,327945	262143	1,237	2048
1024	1,379500	1,355594 a 1,389219	1048575	1,316	8192

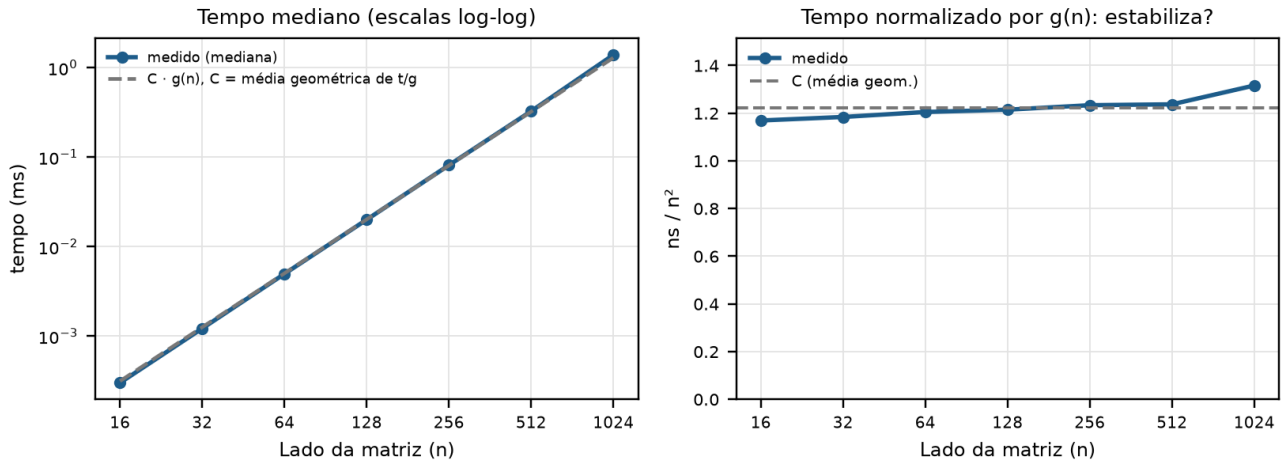


Figura 1. Esquerda: tempo medido, C · g(n). Direita: tempo normalizado, que seria constante se g(n) descrevesse o custo exatamente. Fonte: experimento local (MacBook Apple M4), 28/10/2026.

Convergências

O crescimento é quadrático em todo o intervalo: o expoente empírico é 2,02 (n ≤ 128: 2,02; n ≥ 256: 2,05), contra 2 previsto. Ao dobrar o lado, o tempo multiplica por 4,06 (128 a 256), 4,01 (256 a 512) e 4,26 (512 a 1024), próximo de 4, e a razão ns/g(n) varia pouco, de 1,17 a 1,32 ns. A contagem analítica de comparações, n² - 1 = N - 1, é a de uma varredura linear: a divisão em quadrantes reorganiza o trabalho, mas não reduz a complexidade em função de N, como a teoria prevê (Θ(N)).

Divergências e possíveis causas

O desvio é pequeno e gradual: ns/g(n) cresce 13% de n = 16 a n = 1024, e o maior salto ocorre de n = 512 a n = 1024 (+6%), quando a entrada passa de 2 MiB para 8 MiB; o último dobro fica 6% acima de 4. O Teorema Mestre não modela a hierarquia de memória. Hipóteses, não verificadas nesta coleta: (i) o aumento acompanha o crescimento da entrada (de 2 KiB a 8 MiB), compatível com acessos mais lentos a níveis maiores de memória; (ii) a entrada sempre cabe no L2 do cluster de desempenho (16 MiB), o que é compatível com a ausência de um degrau acentuado; (iii) o macOS decide o núcleo e a frequência de cada lote, e isso não foi controlado. As entradas dos Casos 2 e 3 têm no máximo 8 KiB e não sofrem esse efeito.

As medições descrevem esta faixa de entradas; a prova da complexidade está na seção 4.

5.2 Problema 2 - Soma de vetor com cópias

A tabela traz a mediana das cinco médias de lote, o mínimo e o máximo, a contagem analítica de operações (conferida por instrumentação, seção 3.3) e a razão ns/g(n), com $g(n) = n \cdot \log_3 n$. A unidade ns/g(n) é apenas uma razão de escala, e não o custo exato de uma instrução.

n	Mediana (ms)	Mín. a máx. (ms)	Cópias (fórmula)	ns / g(n)
9	0,000178	0,000176 a 0,000180	18	9,900
27	0,000595	0,000594 a 0,000603	81	7,351
81	0,001856	0,001840 a 0,001861	324	5,729
243	0,005605	0,005555 a 0,005667	1215	4,613
729	0,016982	0,016878 a 0,017146	4374	3,883

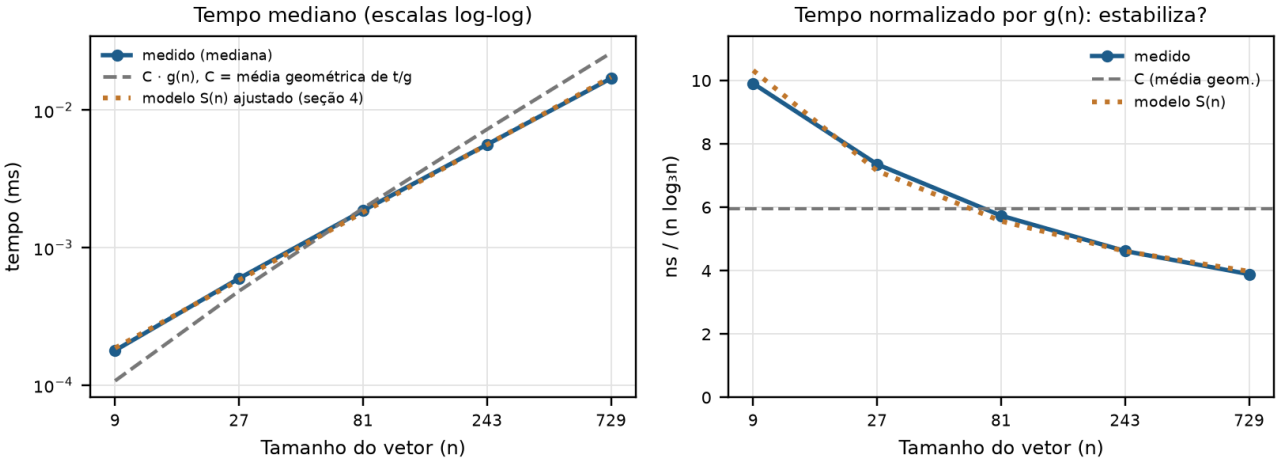


Figura 2. Esquerda: tempo medido, C · g(n) e modelo ajustado. Direita: tempo normalizado, que seria constante se g(n) descrevesse o custo exatamente. Fonte: experimento local (MacBook Apple M4), 28/10/2026.

Convergências

A contagem instrumentada confirma $n \cdot \log_3 n$ cópias, de modo que o termo n log n existe na execução. O tempo cresce 95,30 vezes entre n = 9 e n = 729 (n cresce 81 vezes), valor entre o linear (81) e o previsto para $n \cdot \log_3 n$ (243); o expoente empírico é 1,03, entre 1 (linear) e 1,25 ($n \cdot \log_3 n$ nesta faixa). O modelo da seção 4.2, $S(n) = dn + cn \cdot \log_3 n$, ajustado por mínimos quadrados a $t/n = d + c \cdot \log_3 n$, dá $c \approx 0{,}80$ ns e d efetivo ≈ 19,0 ns (o d efetivo reúne o custo dos casos base e o custo por chamada que não depende do tamanho, como as alocações), com erro máximo de 4,2% nos cinco tamanhos.

Divergências e possíveis causas

A razão ns/g(n) não estabiliza: cai de 9,9 para 3,9, e os fatores por triplicação (3,34, 3,12, 3,02, 3,03) ficam abaixo dos previstos para $n \cdot \log_3 n$ puro, $3(\log_3 n + 1)/\log_3 n$ (4,50, 4,00, 3,75, 3,60). Um modelo de um só parâmetro, $t = C \cdot n \cdot \log_3 n$, erra de -36% a +62%. A causa está no próprio modelo: $S(n)/(n \cdot \log_3 n) = c + d/\log_3 n$ é decrescente e tende a c por cima, exatamente o observado. Como $d/c \approx 24$, o termo linear domina na faixa medida: a árvore tem (3n - 1)/2 chamadas e cada chamada interna faz três malloc e três free, um custo total Θ(n) que o termo linear dn (com o d efetivo) absorve. O termo $cn \cdot \log_3 n$ só passaria a dominar para $\log_3 n > d/c$, fora desta faixa ($\log_3 n$ de 2 a 6). O ajuste usa dois parâmetros e cinco pontos: é compatível com o modelo, mas não o prova; a prova é a da seção 4.2.

As medições descrevem esta faixa de entradas; a prova da complexidade está na seção 4.

5.3 Problema 3 - Distâncias quadráticas entre pares

A tabela traz a mediana das cinco médias de lote, o mínimo e o máximo, a contagem analítica de operações (conferida por instrumentação, seção 3.3) e a razão ns/g(n), com g(n) = n². A unidade ns/g(n) é apenas uma razão de escala, e não o custo exato de uma instrução.

n	Mediana (ms)	Mín. a máx. (ms)	Pares (fórmula)	ns / g(n)
16	0,000050	0,000050 a 0,000051	120	0,196
32	0,000151	0,000150 a 0,000153	496	0,148
64	0,000481	0,000478 a 0,000483	2016	0,117
128	0,001685	0,001669 a 0,001719	8128	0,103
256	0,006837	0,006649 a 0,006893	32640	0,104
512	0,026465	0,026051 a 0,027004	130816	0,101
1024	0,100961	0,099227 a 0,102828	523776	0,096

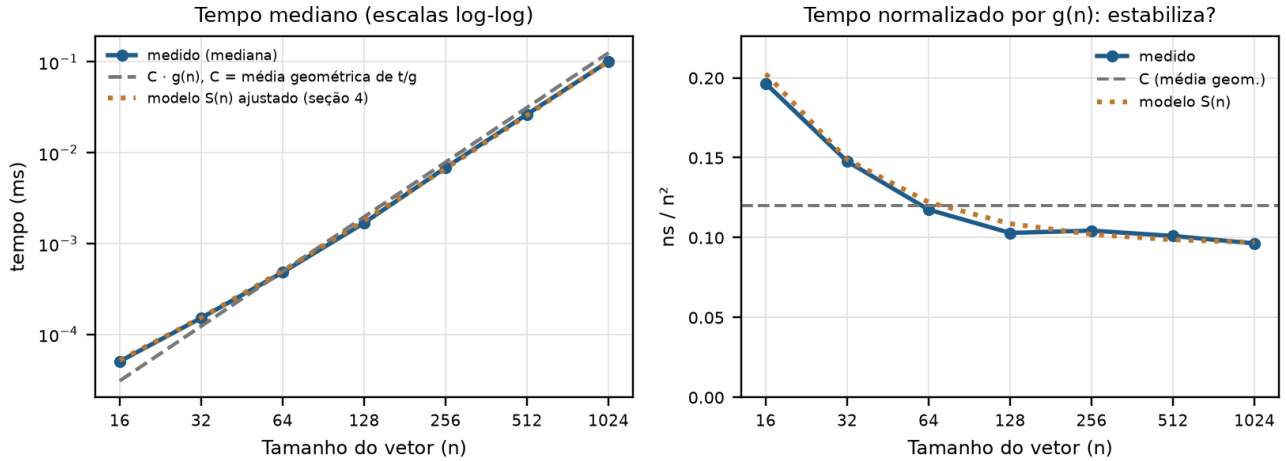


Figura 3. Esquerda: tempo medido, C · g(n) e modelo ajustado. Direita: tempo normalizado, que seria constante se g(n) descrevesse o custo exatamente. Fonte: experimento local (MacBook Apple M4), 28/10/2026.

Convergências

O expoente empírico é 1,97 para n ≥ 128 (previsto: 2), e a razão ns/n² estabiliza perto de 0,096 ns (cerca de 0,19 ns por par avaliado, pois há n(n - 1)/2 pares). Nos três últimos dobros, o tempo multiplica por 4,06, 3,87 e 3,81, próximo de 4. O modelo da seção 4.3, S(n) = 2cn² + (d - 2c)n, ajustado a t/n = 2c·n + (d - 2c), dá 2c ≈ 0,10 ns e d - 2c ≈ 1,7 ns, com erro máximo de 5,6% para n ≥ 64. A entrada (no máximo 8 KiB) cabe no L1 de dados (64 KiB).

Divergências e possíveis causas

Nos tamanhos pequenos, ns/n² é 0,20 para n = 16, cerca de 104% acima do valor para n = 1024, e as razões por dobra são menores que 4 (3,01 de 16 para 32). A causa está no termo linear da fórmula provada: como d - 2c > 0, os custos fixos e das chamadas recursivas pesam mais quando n é pequeno e a razão tende a 2c por cima. O desvio é, portanto, o previsto pelo modelo, e não uma contradição do Caso 3. O modelo dá razões por dobra menores que 4; o dobro de 128 para 256 (4,06) fica pouco acima, o que pode vir da variabilidade do ambiente.

As medições descrevem esta faixa de entradas; a prova da complexidade está na seção 4.

6. Conclusão

6.1 Resumo dos principais resultados

Problema	a, b, p	f(n)	Caso	Θ (demonstrado)	Expoente empírico
1. Máximo em matriz	4, 2, 2	Θ(1)	1	Θ(n²) = Θ(N)	2,02 (n ≥ 16)
2. Soma com cópias	3, 3, 1	Θ(n)	2	Θ(n log n)	1,03 (n log n: 1,25 nesta faixa)
3. Distâncias	2, 2, 1	Θ(n²)	3	Θ(n²)	1,97 (n ≥ 128)

Os três problemas atendem aos casos solicitados e às restrições do enunciado. As demonstrações por indução, com a passagem do modelo S para T, confirmam os resultados do Teorema Mestre, e a validação (102 verificações e as contagens instrumentadas) confirma a correção dos programas e das fórmulas de contagem.

6.2 Síntese da comparação entre teoria e prática

Nos Casos 1 e 3, os tempos acompanham o crescimento quadrático previsto; no Caso 1 o desvio é pequeno e gradual (ns/g(n) varia 13% em todo o intervalo), e no Caso 3 os desvios em n pequeno são explicados pelo termo linear do modelo. No Caso 2, a faixa medida não evidencia isoladamente o fator logarítmico, e o ajuste de $dn + cn \cdot \log_3 n$ explica o comportamento: o custo linear de chamadas e alocações domina entre n = 9 e n = 729. A experiência apoia as previsões onde a faixa permite e registra, onde não permite, a causa provável, sem tratar a proximidade das curvas como prova.

6.3 Observações do desenvolvimento

Dividir uma entrada em subproblemas não garante, por si só, menor complexidade: importam o número de chamadas e o custo de combinação. O Problema 1 faz N - 1 comparações, como uma varredura, e o Problema 2 mostra que o mesmo esquema de três chamadas passa do Caso 2 ao Caso 1 se as cópias forem retiradas. No Problema 1, n é o lado e a entrada tem n² elementos; nos outros, n é a quantidade de elementos, e por isso as curvas não devem ser comparadas como se as entradas tivessem o mesmo tamanho. Também não é correto ordenar algoritmos de problemas distintos pelo tempo absoluto: as contagens de comparações, cópias e pares são unidades diferentes.

6.4 Limitações

Faixa restrita de tamanhos (sobretudo no Caso 2), uma única máquina (coleta na bateria, com núcleo e frequência não controlados), cinco amostras por tamanho, uma entrada pseudoaleatória por tamanho e reutilização de dados nos lotes. Não se mediu consumo real de memória, e as hipóteses sobre cache e predição de desvios não foram testadas.

6.5 Considerações adicionais

O Problema 3 enumera pares em Θ(n²), mas pela identidade $D = n\Sigma v^2 - (\Sigma v)^2$ o problema se resolve em Θ(n); a escolha do algoritmo, e não do problema, determina o caso do teorema. Como continuação, vale estender a faixa do Caso 2 (por exemplo, até $n = 3^{12}$), repetir o experimento com núcleo e frequência controlados, ou em outra máquina, para testar a hipótese de cache no Caso 1, e comparar as versões recursivas com as iterativas.

Referências

[1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 4. ed. MIT Press, 2022. Capítulo 4 (divisão e conquista e método mestre).

Apêndice A - Dados experimentais (CSV completo)

Arquivo results/experimentos.csv: 95 amostras, tempo_ms em milissegundos por execução (média do lote), repeticoes = tamanho do lote.

Por falta de espaço, as linhas de dados são distribuídas em duas colunas (a primeira metade à esquerda, a segunda à direita), e o cabeçalho se repete no topo de cada coluna, quebrado em duas linhas. No arquivo CSV há uma única tabela, com o cabeçalho em uma linha e as linhas de dados em sequência.

```
caso,n,amostra,repeticoes,tempo_ms,
  metrica_exata,referencia_assintotica,validado
1,16,1,65536,0.000383774,255,256.000000000,1
1,16,2,131072,0.000298927,255,256.000000000,1
1,16,3,131072,0.000300934,255,256.000000000,1
1,16,4,131072,0.000299240,255,256.000000000,1
1,16,5,131072,0.000293831,255,256.000000000,1
1,32,1,32768,0.001204468,1023,1024.000000000,1
1,32,2,32768,0.001221222,1023,1024.000000000,1
1,32,3,32768,0.001212097,1023,1024.000000000,1
1,32,4,32768,0.001215027,1023,1024.000000000,1
1,32,5,32768,0.001209717,1023,1024.000000000,1
1,64,1,8192,0.004825928,4095,4096.000000000,1
1,64,2,8192,0.004980957,4095,4096.000000000,1
1,64,3,8192,0.004933716,4095,4096.000000000,1
1,64,4,8192,0.004980469,4095,4096.000000000,1
1,64,5,8192,0.004877930,4095,4096.000000000,1
1,128,1,2048,0.020106934,16383,16384.000000000,1
1,128,2,2048,0.019732422,16383,16384.000000000,1
1,128,3,2048,0.019677734,16383,16384.000000000,1
1,128,4,2048,0.028858887,16383,16384.000000000,1
1,128,5,2048,0.019893555,16383,16384.000000000,1
1,256,1,512,0.080808594,65535,65536.000000000,1
1,256,2,512,0.081951172,65535,65536.000000000,1
1,256,3,512,0.080751953,65535,65536.000000000,1
1,256,4,512,0.081546875,65535,65536.000000000,1
1,256,5,512,0.080654297,65535,65536.000000000,1
1,512,1,128,0.324203125,262143,262144.000000000,1
1,512,2,128,0.325234375,262143,262144.000000000,1
1,512,3,128,0.323000000,262143,262144.000000000,1
1,512,4,128,0.322515625,262143,262144.000000000,1
1,512,5,128,0.327945313,262143,262144.000000000,1
1,1024,1,32,1.388875000,1048575,1048576.000000000,1
1,1024,2,32,1.379500000,1048575,1048576.000000000,1
1,1024,3,32,1.389218751,1048575,1048576.000000000,1
1,1024,4,32,1.355593749,1048575,1048576.000000000,1
1,1024,5,32,1.362312501,1048575,1048576.000000000,1
2,9,1,262144,0.000178741,18,18.000000000,1
2,9,2,262144,0.000178207,18,18.000000000,1
2,9,3,262144,0.000179588,18,18.000000000,1
2,9,4,262144,0.000176346,18,18.000000000,1
2,9,5,262144,0.000176952,18,18.000000000,1
2,27,1,65536,0.000593887,81,81.000000000,1
2,27,2,65536,0.000601242,81,81.000000000,1
2,27,3,65536,0.000595413,81,81.000000000,1
2,27,4,65536,0.000603363,81,81.000000000,1
2,27,5,65536,0.000594238,81,81.000000000,1
2,81,1,16384,0.001861389,324,324.000000000,1
2,81,2,16384,0.001839905,324,324.000000000,1
2,81,3,16384,0.001858215,324,324.000000000,1
```

```
caso,n,amostra,repeticoes,tempo_ms,
  metrica_exata,referencia_assintotica,validado
2,81,4,16384,0.001841736,324,324.000000000,1
2,81,5,16384,0.001856323,324,324.000000000,1
2,243,1,8192,0.005604614,1215,1215.000000000,1
2,243,2,8192,0.005667358,1215,1215.000000000,1
2,243,3,8192,0.005611816,1215,1215.000000000,1
2,243,4,8192,0.005588257,1215,1215.000000000,1
2,243,5,8192,0.005555054,1215,1215.000000000,1
2,729,1,2048,0.016982422,4374,4374.000000000,1
2,729,2,2048,0.017146484,4374,4374.000000000,1
2,729,3,2048,0.016910645,4374,4374.000000000,1
2,729,4,2048,0.016997070,4374,4374.000000000,1
2,729,5,2048,0.016878418,4374,4374.000000000,1
3,16,1,524288,0.000050444,120,256.000000000,1
3,16,2,524288,0.000051018,120,256.000000000,1
3,16,3,524288,0.000050024,120,256.000000000,1
3,16,4,524288,0.000050280,120,256.000000000,1
3,16,5,524288,0.000049557,120,256.000000000,1
3,32,1,262144,0.000150433,496,1024.000000000,1
3,32,2,262144,0.000149952,496,1024.000000000,1
3,32,3,262144,0.000152885,496,1024.000000000,1
3,32,4,262144,0.000151169,496,1024.000000000,1
3,32,5,262144,0.000152275,496,1024.000000000,1
3,64,1,65536,0.000478088,2016,4096.000000000,1
3,64,2,65536,0.000482452,2016,4096.000000000,1
3,64,3,65536,0.000479965,2016,4096.000000000,1
3,64,4,65536,0.000482635,2016,4096.000000000,1
3,64,5,65536,0.000480988,2016,4096.000000000,1
3,128,1,16384,0.001685242,8128,16384.000000000,1
3,128,2,16384,0.001690002,8128,16384.000000000,1
3,128,3,16384,0.001668518,8128,16384.000000000,1
3,128,4,16384,0.001678162,8128,16384.000000000,1
3,128,5,16384,0.001718628,8128,16384.000000000,1
3,256,1,4096,0.006878418,32640,65536.000000000,1
3,256,2,4096,0.006774902,32640,65536.000000000,1
3,256,3,4096,0.006892578,32640,65536.000000000,1
3,256,4,4096,0.006648682,32640,65536.000000000,1
3,256,5,4096,0.006836670,32640,65536.000000000,1
3,512,1,1024,0.026632813,130816,262144.000000000,1
3,512,2,1024,0.027003906,130816,262144.000000000,1
3,512,3,1024,0.026464844,130816,262144.000000000,1
3,512,4,1024,0.026434570,130816,262144.000000000,1
3,512,5,1024,0.026050781,130816,262144.000000000,1
3,1024,1,256,0.100960938,523776,1048576.000000000,1
3,1024,2,256,0.102828125,523776,1048576.000000000,1
3,1024,3,256,0.101687500,523776,1048576.000000000,1
3,1024,4,256,0.099226562,523776,1048576.000000000,1
3,1024,5,256,0.100925781,523776,1048576.000000000,1
```

Apêndice B - Parâmetros da execução

Para reproduzir as medições com o mesmo critério: gerador congruencial linear determinístico com semente 20261028 e valores inteiros em [-10, 10]; tamanhos n = 16 a 1024 (potências de 2) nos Casos 1 e 3 e n = 9 a 729 (potências de 3) no Caso 2; cinco amostras por tamanho; lote com duração mínima de 25 ms e no máximo 1.048.576 repetições; compilação com Apple clang 21.0.0, opções -O2 -Wall -Wextra -pedantic, em macOS 27.0.1 (Apple M4). O CSV é sobrescrito a cada execução, e novas medições podem variar conforme o estado da máquina.

Os valores gerados estão em [-10, 10] e os resultados usam int64_t; os limites não devem ser aumentados sem verificar overflow. O campo validado = 1 informa que o resultado daquele tamanho passou pela referência antes dos lotes. As tabelas e os gráficos deste relatório são gerados a partir do CSV por um script em Python (ReportLab e Matplotlib).